

BACKEND MODULE DESIGN DOCUMENT

PART-3: RUNTIME BEHAVIOR STANDARDS

School ERP

Transactions | Caching | Events | Background Jobs | Integrations

Prepared by: Backend Architecture Team

Document Version: 1.0

Date: 29 July 2026

Classification: Confidential — Engineering Team Only

Preamble

This document is Part-3 of the Backend Module Design Document (BMDD) v1.0 for the School ERP System, continuing directly from Part-1: Backend Module Architecture and Part-2: Entity, Repository, Service & Controller Standards. It defines backend runtime-behavior standards — transaction propagation, caching, event processing, background jobs, external integrations, file processing, concurrency, and performance optimization — consolidating MDB Part-3, DDD Parts 1-4, ASD Parts 1-4, and BMDD Parts 1-2 into concrete implementation guidance.

This document reuses RGD v2.0, Appendix A-N, the UI/UX Screen Specifications, MDB Parts 1-4, DDD Parts 1-4, ASD Parts 1-4, and BMDD Parts 1-2 by reference only, and does not regenerate any of them. It contains no source code and no implementation — every standard is stated as a rule only.

21. Transaction Propagation

Extends the transaction-boundary rule already fixed in BMDD Part-2 Section 17 with explicit propagation behavior for nested Service calls.

- A Service method invoked from within another Service method's transaction (same module, or a cross-module published-interface call per BMDD Part-1 Section 6) participates in that existing transaction by default — a nested call does not silently start an independent transaction unless explicitly required.
- An independent transaction is used only where the nested operation must commit regardless of the outer operation's outcome — this is a documented, deliberate exception (e.g., an AuditLogService write that must survive even if the enclosing business operation later rolls back for an unrelated reason within the same request), not a default pattern.
- Event publication (BMDD Part-1 Section 9, Section 24 this document) occurs only after the publishing transaction has committed — an event is never raised from within a still-open transaction that might yet roll back, preventing a consumer from acting on a change that never actually happened.
- A read-only nested call (e.g., a Service method checking an eligibility condition before a write proceeds) shares the outer transaction's read view rather than opening a separate connection, avoiding unnecessary connection-pool pressure (MDB Part-3 Section 21) within a single logical operation.

22. Caching

Fixes the Service-layer implementation of the Redis caching strategy already established in MDB Part-3 Section 26.

- Cache reads and writes are implemented in the Service layer only (BMDD Part-2 Section 13) — a Repository or Controller never interacts with Redis directly, keeping the cache-aside pattern's read-through/write-through logic in one consistent location per module.
- Cache keys follow a consistent, module-prefixed naming convention (e.g., a module and entity identifier combined with the entity's surrogate ID) so that keys from different modules never collide and a key's origin is identifiable during troubleshooting.
- Time-to-live is set per the classification already fixed in DDD Part-2 Section 11 — Master Data entries use a longer TTL consistent with their low change-frequency, while any inherently time-bounded cached value (e.g., a published Timetable for the current term) expires no later than that value's own natural validity period.
- A cache read failure (Redis temporarily unavailable) falls back to the database read transparently — caching is a performance optimization, never a hard dependency for correctness; a Service method never fails solely because the cache layer is unreachable.

23. Cache Invalidation

Extends MDB Part-3 Section 26's invalidation-trigger rule with the specific implementation discipline preventing stale-cache leaks across module boundaries.

- Every Service method that writes to a cache-eligible entity (Section 22) explicitly evicts or updates the corresponding Redis key within the same method, before returning — invalidation is never deferred to a background sweep or time-based expiry alone for correctness-sensitive Master Data.
- A cross-module write that affects another module's cached data (e.g., a Class capacity change affecting a cached Route/Section-derived value) explicitly invalidates the dependent key as well, identified at design time per entity relationship (DDD Part-2 Section 17) rather than discovered later as a stale-data defect.
- Bulk operations (Bulk Marks Upload, Bulk Fee Collection, Bulk Attendance Entry — per the respective UI/UX Screen Specifications) invalidate the affected cache keys as a single batch operation at the end of the bulk write, not once per individual row, avoiding an unnecessary invalidation storm.
- A cache invalidation failure (Redis temporarily unavailable at the moment of invalidation) is logged and does not fail the underlying database write — the design instead relies on the fixed TTL (Section 22) as a bounded staleness backstop for this rare failure case.

24. Event Processing

Extends the Event Publisher/Consumer pattern already fixed in BMDD Part-1 Section 9 with concrete processing guarantees.

- Events are delivered at-least-once, not exactly-once — every event consumer is implemented to be idempotent, safely handling a duplicate delivery of the same event without producing an incorrect duplicate effect (e.g., a Fee posting triggered twice by a redelivered Library-fine event must not post the fine twice).
- Event ordering is not guaranteed across different event types, but is preserved for events of the same type from the same source, consistent with the eventually-consistent nature already fixed for cross-module effects in BMDD Part-1 Section 9.
- A consumer that fails to process an event retries per the Background Job retry pattern (Section 25 this document, Appendix-J Section 17) rather than silently dropping the event — a repeatedly-failing event is escalated for manual review rather than retried indefinitely without limit.
- Event payloads carry only the identifiers and minimal data a consumer needs to re-fetch current state via the publishing module's Service interface (BMDD Part-1 Section 6) — an event payload is not treated as a durable snapshot of data that might change before the consumer processes it.

25. Background Jobs

Fixes Spring-Boot-specific implementation standards for the 8 background/scheduled processes already catalogued in Appendix-J Section 17.

- Every scheduled job runs on exactly one Application Layer instance at a time even when the backend is horizontally scaled (MDB Part-1 Section 4) — a distributed-scheduling safeguard prevents the same job from executing redundantly across multiple instances simultaneously.
- A background job's business logic resides in the owning module's Service layer (BMDD Part-2 Section 13), invoked by a thin scheduling trigger — the job trigger itself contains no business logic, preserving the same layer-responsibility discipline already fixed in BMDD Part-1 Section 5.
- A failed job execution retries according to a bounded retry policy before being escalated to Infrastructure/Operations (Appendix-N Section 26/MDB Part-4 Section 39) — a job is never left silently failing without eventual escalation.
- Long-running jobs (large report generation, bulk report-card generation) follow the background-job pattern already fixed for large exports (Appendix-F NFR-RPTP-001, ASD Part-3 Section 27), completing asynchronously with a completion notification rather than holding an API request open.

26. External Integrations

Fixes Spring-Boot-specific implementation standards for the external integrations already catalogued in Appendix-I Section 11 and Appendix-K Section 17.

- Every external integration (Payment Gateway, SMS/Email Gateway, GPS, UIDAI, Tally, SSO) is implemented behind a dedicated adapter component in the administration package (BMDD Part-1 Section 6) — a module's own Service layer never calls an external vendor's interface directly, isolating vendor-specific detail from business logic.
- Every integration call carries an explicit timeout and a bounded retry policy, consistent with the per-integration independent monitoring already fixed in Appendix-F NFR-MON-002 and MDB Part-4 Section 37 — an integration call never blocks a Service method indefinitely.
- An integration failure is handled per the Integration Exception category already fixed in BMDD Part-2 Section 16, with the calling Service method's own transaction (Section 21 this document) never spanning the external call itself — the local operation and the external effect are reconciled via the Event Publisher/retry pattern (Sections 24-25), not a distributed transaction.
- A repeated, sustained failure of a specific integration is detected and reported independently of other integrations (per-channel isolation, Appendix-I Section 11.2), so a Payment Gateway outage does not obscure or get conflated with an SMS Gateway outage in monitoring (MDB Part-4 Section 37).

27. File Processing

Fixes Spring-Boot-specific implementation standards for file upload, malware scanning, and generated-document production, extending MDB Part-2 Section 18 and ASD Part-3 Section 27.

- An uploaded file is held in temporary storage and scanned before a Document record (DDD Part-1/2) is created — the Service method handling upload does not return a success response until the scan completes, consistent with ASD Part-3 Section 27.
- Generated-document production (report cards, receipts, payslips, ID cards, Transfer Certificates) is implemented via JasperReports (MDB Part-1 Section 3), invoked from the owning module's Service layer, producing a Document record identical in structure to an uploaded file from that point forward — the two paths converge into the same Document-handling convention (DDD Parts 1-2) after generation/upload completes.
- Large or batch document generation (bulk Report Card Generation, per the Examination UI/UX Screen Specification) follows the Background Job pattern (Section 25 this document) rather than generating synchronously within an API request.
- File retrieval is always mediated through an authorized Service-layer call (Appendix-I Section 12.3, ASD Part-2 Section 12) — a Controller never exposes a raw, directly-addressable storage path in any response.

28. Concurrency

Fixes backend concurrency handling, extending MDB Part-3 Section 25 and DDD Part-3's optimistic/pessimistic locking standards.

- Every Service component is stateless (BMDD Part-1 Section 2) — no Service holds mutable instance state between requests, so a single Service bean instance safely serves many concurrent requests without cross-request interference.
- Optimistic locking (the version column, DDD Part-1 Section 7) is the default concurrency-control mechanism for concurrent updates to the same row — a Concurrency Exception (BMDD Part-2 Section 16) is the expected, handled outcome of a genuine conflict, not a defect.
- Pessimistic locking is used only for the specific, safety-critical cases already identified in MDB Part-3 Section 25 (e.g., BR-ADM-001/BR-ADM-007 seat-allocation capacity) — it is never adopted as a general-purpose default, since broad pessimistic locking would undermine the horizontal-scalability goal already fixed in MDB Part-1 Section 4.
- A concurrent request retry (client-side, after receiving a Concurrency Exception) re-fetches current state before resubmitting — the backend never assumes a retried request's payload reflects the latest state without that explicit re-fetch.

29. Performance Optimization

Consolidates the backend-implementation-level performance practices already fixed across DDD Part-3 Section 27, MDB Part-3 Section 27, and ASD Part-3 Section 35 into a single operational reference for Service and Repository implementation.

- Lazy fetch remains the default for every JPA relationship (DDD Part-1 Section 7); a Service method needing related data across a relationship fetches it explicitly and intentionally, avoiding uncontrolled N+1 query patterns, particularly around the high-fan-out Student and Employee entities.
- Batch database operations (Section 25's bulk-upload screens) use JPA/Hibernate batch-insert/update configuration rather than a per-row persistence call inside a loop, consistent with DDD Part-3 Section 27.
- Caching (Sections 22-23) is applied first to the highest-read-frequency Master Data before any lower-level query-tuning effort, since a well-placed cache typically yields a larger performance gain than incremental query optimization on the same data.
- Every new Repository method touching a high-volume table is reviewed against an execution plan before merge (DDD Part-3 Section 27, Section 30 this document) — this review is not deferred to a later, dedicated performance-tuning pass.

30. Backend Performance Checklist

Verified for every new or changed backend component with runtime-behavior impact, complementing the Backend Quality Checklist already fixed in BMDD Part-2 Section 20.

#	Checklist Item
1	Nested Service calls (Section 21) participate in the correct transaction, with any independent-transaction exception explicitly documented
2	Cache reads/writes (Section 22) occur only in the Service layer, with a documented fallback on cache unavailability
3	Every write to a cache-eligible entity includes explicit invalidation (Section 23), including any cross-module dependent key
4	Event consumers (Section 24) are idempotent against at-least-once, possibly duplicate, delivery
5	Background jobs (Section 25) run singly across scaled instances and escalate on repeated failure
6	External integration calls (Section 26) carry an explicit timeout and bounded retry, isolated per integration
7	File operations (Section 27) scan before success and never expose a raw storage path
8	Concurrency control (Section 28) uses optimistic locking by default, pessimistic only for the documented safety-critical cases
9	New high-volume queries (Section 29) are reviewed against an execution plan before merge